PROMPTVAULT

Product Requirements Document

Privacy-first security firewall for generative AI prompts

Document field	Value
Version	1.0
Prepared	18 October 2026
Product strategy	Standalone web application first; browser extension later
Target	Hackathon MVP built on a production-ready foundation
Core constraint	Core protection must work without paid AI APIs
Model strategy	Optional local/open-weight model evaluation, including eligible NVIDIA models
Excluded content	UI/UX specifications, screen layouts, visual design, and interaction-design recommendations

Status: Proposed baseline for implementation planning. Requirements and numeric targets should be validated against the selected implementation, hardware, and benchmark dataset before making public security claims.

Document guide

- 1. Executive summary
- 2. Product vision, problem, and principles
- 3. Goals, non-goals, and scope
- 4. Target users and use cases
- 5. Product requirements and priorities
- 6. Detection taxonomy and engine specification
- 7. Risk scoring, policy decisions, and redaction
- 8. Privacy, data lifecycle, and retention
- 9. System architecture and technology strategy
- 10. Security threat model and abuse cases
- 11. API and internal contracts
- 12. Non-functional requirements
- 13. Testing, evaluation, and acceptance criteria
- 14. Metrics and success criteria
- 15. Deployment, operations, and governance
- 16. Roadmap and workstreams
- 17. Hackathon demonstration plan
- 18. Risks, assumptions, dependencies, and open decisions
- Appendix A. Sample policy and sample findings
- Appendix B. Glossary

This PRD specifies product behavior and engineering constraints. It intentionally excludes UI/UX design. Any implementation details below are recommendations where stated as such; the acceptance criteria define the required outcomes.

1. Executive summary

PromptVault is a privacy-first security layer that detects sensitive information in prompts before users send those prompts to generative AI services. It identifies potential exposure of personal information, credentials, cryptographic material, sensitive code patterns, and organization-defined confidential terms. It assigns findings severity and confidence, explains the basis for detection, and supports deterministic redaction or blocking under policy.

The first release is a standalone web application. A browser extension is a later phase that will reuse the security core and apply it to supported AI websites. The initial implementation must be modular enough that browser-specific enforcement does not require rewriting the detector, policy engine, or redaction logic.

Core protection must run without paid AI APIs. Deterministic pattern matching, validators, structured parsers, and configurable rules form the baseline. Optional local models—including suitable NVIDIA open models—may be evaluated for contextual classification, but they must not be the sole security control. Model choice is conditional on license review, benchmark performance, resource requirements, and local deployment feasibility.

Security promise: PromptVault reduces accidental disclosure for supported content and tested paths. It cannot guarantee detection of every sensitive fact, secure behavior by third-party AI providers, or protection on unsupported submission paths.

1.1 Product summary

Area	Requirement summary
Primary deliverable	Standalone web application that scans, classifies, sanitizes, and validates text locally.
Later deliverable	Chromium-based browser extension with supported-site adapters and submission interception.
Core security functions	Detection, confidence/severity classification, policy evaluation, redaction, post-redaction validation.
Privacy default	Original prompt content and raw findings remain in transient local processing; no server transmission by default.
Model policy	Optional local inference behind an interface; deterministic fallback remains available.
Initial target users	Students, developers, individual AI users, startups, and security teams.
Initial release evidence	Repeatable test suite, documented limitations, benchmark report, and synthetic attack-defense demonstration.

1.2 Definition of success

The project succeeds when it can demonstrate repeatable detection and redaction on a documented synthetic dataset, avoids transmitting original prompts during default local operation, prevents unsafe output from being marked safe without post-redaction checks, and has a clear path from the web MVP to browser enforcement.

2. Product vision, problem, and principles

2.1 Vision

Make AI adoption safer and more transparent by placing a lightweight, explainable privacy control between a user and the AI service they intend to use.

2.2 Problem statement

Users frequently paste text into generative AI tools that contains secrets or confidential information embedded in otherwise ordinary requests. Examples include source code with credentials, database connection strings, customer records, personal identifiers, internal documents, logs, and configuration files. Users may not notice these values, and generic keyword checks do not reliably distinguish sensitive content from harmless text.

This creates risks of credential theft, privacy violations, exposure of intellectual property, and noncompliance with organizational policies. PromptVault aims to identify likely exposures before submission, explain the relevant risk, and provide a safe remediation path.

2.3 Principles

- **Privacy by default:** ordinary scanning and redaction should run locally; network access must not be necessary for core protection.
- **Prevention over notification:** strict policies must block unresolved prohibited content rather than merely warn.
- **Deterministic baseline:** essential security behavior must work without a language model.
- **Meaning-aware but conservative:** preserve useful context where possible; do not claim semantic equivalence is guaranteed.
- **Explainability:** findings carry categories, confidence, severity, detector identifiers, and reason codes.
- **Data minimization:** do not persist raw prompts, raw secrets, or reversible mappings by default.
- **Least privilege:** the later extension accesses only supported AI sites and only the content needed for protection.
- **Honest guarantees:** document false negatives, unsupported paths, model limitations, and third-party boundaries.
- **Reproducibility:** rules, policies, model versions, and benchmark datasets are versioned.

3. Goals, non-goals, and scope

3.1 Goals

- Detect supported PII, credentials, secrets, structured connection strings, and configured sensitive terms.
- Sanitize selected findings with typed, consistent placeholders while preserving unrelated text.
- Evaluate findings against a deterministic and configurable policy engine.
- Re-scan sanitized output and report residual findings and uncertainty.
- Provide an auditable, synthetic attack-and-defense test workflow.
- Keep the initial product functional without paid AI APIs or mandatory hosted inference.
- Prepare a shared security core for future browser extension reuse.
- Establish production-oriented engineering practices from the start.

3.2 Non-goals

- Guaranteeing that every possible secret or confidential statement will be detected.
- Replacing enterprise DLP, endpoint detection, identity management, or legal compliance review.
- Guaranteeing that an AI provider does not retain or train on content submitted to that provider.
- Automatically determining the confidentiality of all source code or business prose without context.
- Inspecting unrelated local files, unrelated browser forms, browsing history, or cookies.
- Testing the validity of credentials against third-party services.
- Automatically uploading original prompts to a cloud model for analysis.
- Supporting all AI websites or all browser submission pathways in the first extension release.

3.3 Release scope

Stage	Included	Exit condition
Stage 1: Web MVP	Shared security core, text scanning, deterministic rules, policy evaluation, redaction, local validation, synthetic attack suite, privacy checks.	Core flows pass tests; benchmark results and known limitations are documented.
Stage 2: Hardening	Adversarial testing, dependency/security scans, input limits, robust errors, retention verification, performance checks, release process.	No unresolved critical/high findings in agreed release gate; privacy claims verified.
Stage 3: Extension	Manifest V3 extension, supported-site adapters, pre-submission interception, strict/permissive policies, adapter health tests.	Each supported site passes explicit interception and bypass tests.
Stage 4: Team features	Optional policy distribution, authenticated configuration, aggregate reporting, managed deployment.	Threat model and access controls reviewed before organizational rollout.

4. Target users and use cases

Audience	Primary need	Example
Students and educators	Avoid sharing personal data or credentials while asking for help.	A programming assignment includes a real API token.
Developers	Detect secrets in code, logs, configuration, and error traces.	A database password appears in a connection string.
Startups and businesses	Apply policy to customer information and confidential internal data.	A user pastes a customer record into a summarization prompt.
Security teams	Measure coverage, false positives, and prevention behavior.	A team runs synthetic leak scenarios against policy rules.
Individual AI users	Understand and reduce unnecessary disclosure.	A prompt includes an email address that can be replaced with a placeholder.

4.1 Core scenarios

1. A user scans a prompt containing a recognizable API key. PromptVault identifies the range, assigns a high-severity category, and proposes redaction.
2. A user asks for help with a connection string. PromptVault detects the embedded credential and offers a sanitized form that preserves safe context where feasible.
3. A prompt includes an email address. The system flags it according to policy and allows a permitted retain or redact decision.
4. A prompt includes an ambiguous token-like string. The detector reports uncertainty instead of asserting the string is a confirmed secret.
5. A detector fails or times out. The system reports the scan failure; it must not silently mark the input safe.
6. A synthetic attack suite runs multiple test cases. Results distinguish true positives, false positives, false negatives, and unresolved cases.
7. In a future extension, a user submits a prompt on a supported AI website. The extension inspects the actual outgoing text and blocks or sanitizes it according to policy.

5. Product requirements and priorities

Priority definitions: P0 = core release requirement; P1 = production-hardening or first general-release requirement; P2 = later enhancement. A requirement is not complete merely because a code path exists; it must have a test or other verification evidence.

5.1 Scanning and detection

ID	Requirement	Priority
FR-001	Accept supported text input for inspection without an external AI API.	P0
FR-002	Detect supported personal information using patterns and contextual methods.	P0
FR-003	Detect supported credential and secret formats.	P0
FR-004	Return category, exact span, severity, confidence, detector ID, and reason code for each finding.	P0
FR-005	Handle multiple findings in one prompt.	P0
FR-006	Resolve duplicate and overlapping findings deterministically.	P0
FR-007	Support validated, versioned detection rules.	P1
FR-008	Support optional contextual classification through a pluggable local model.	P1
FR-009	Distinguish confidence from severity.	P0
FR-010	Report scanner failures explicitly; never silently return a safe result on failure.	P0

5.2 Sanitization and redaction

ID	Requirement	Priority
FR-011	Redact a selected finding without changing unrelated text.	P0
FR-012	Support redaction of all policy-eligible findings in one operation.	P0
FR-013	Support typed placeholders and consistent replacement of repeated values.	P0
FR-014	Preserve punctuation, line breaks, and code structure where practical.	P0
FR-015	Require validation of sanitized output before it is marked policy-compliant.	P0
FR-016	Support explicit retain/allow decisions subject to mandatory policy.	P1
FR-017	Provide optional local restoration mapping with strict safeguards.	P1
FR-018	Never automatically submit the original prompt after redaction.	P0
FR-019	Reject stale, malformed, or overlapping ranges that cannot be reconciled safely.	P0
FR-020	Report residual findings and unresolved semantic risk.	P0

5.3 Policy and risk

ID	Requirement	Priority
FR-021	Compute a versioned risk assessment from findings and configured rules.	P0
FR-022	Support ALLOW, WARN, REDACT, BLOCK, and REVIEW decisions.	P0
FR-023	Return the reason codes and mandatory rules behind each decision.	P0
FR-024	Support user-level policies and a future authenticated organization policy.	P1
FR-025	Ensure mandatory rules cannot be bypassed by ordinary user overrides.	P1
FR-026	Separate simulation events from actual submission events.	P0

5.4 Privacy and observability

ID	Requirement	Priority
FR-027	Keep original prompt text and raw sensitive values out of persistent logs by default.	P0
FR-028	Provide privacy-safe aggregate counts and decision outcomes.	P1
FR-029	Keep core scanning operational when optional reporting services are unavailable.	P0
FR-030	Document all network egress and verify default local-only behavior.	P0
FR-031	Version detector rules, policy schemas, and optional model configurations.	P1

6. Detection taxonomy and engine specification

6.1 Classification taxonomy

Category	Examples	Approach and limitation
Personal information	Email, phone, names in context	Regex and contextual NER; names can be ambiguous.
Government identifiers	Supported national or tax IDs	Jurisdiction-specific formats and validators; avoid claiming universal coverage.
Financial information	Payment card/account-like values	Pattern and checksum validation where applicable.
Authentication secrets	API keys, passwords, bearer tokens	Known signatures, contextual labels, entropy heuristics.
Cryptographic material	Private-key blocks, seed phrases	Structural signatures and dedicated validators where available.
Database credentials	Connection strings with embedded secrets	Structured parsing and credential-field detection.
Cloud credentials	Provider-specific keys and tokens	Maintained provider patterns and test fixtures.
Confidential code	Restricted implementation details	Explicit markers and organization rules; arbitrary proprietary code is not reliably inferable.
Internal business data	Customer IDs, project codes, restricted labels	Organization-defined patterns and dictionaries.
Health-related data	Supported medical identifiers and records	Contextual detection and configured policies; not a medical diagnosis.
Custom sensitive data	Internal labels and unique identifiers	User/organization rules with validation and versioning.

6.2 Pipeline

1. **Input validation:** enforce size and type limits, validate encoding, and reject malformed requests.
2. **Normalization:** apply carefully selected Unicode and whitespace handling while preserving a mapping to original offsets.
3. **Deterministic detection:** run regex rules, structured parsers, validators, secret signatures, and configured patterns.
4. **Contextual detection:** optionally run local NER or a local open model for ambiguous context.
5. **Finding reconciliation:** merge duplicates, resolve overlaps deterministically, and preserve provenance.
6. **Confidence/severity assessment:** attach independent confidence and impact estimates.
7. **Policy evaluation:** evaluate mandatory rules and return a decision plus reason codes.
8. **Sanitization:** replace selected spans using a deterministic algorithm.

9 **Post-redaction validation:** re-scan the sanitized text and confirm mandatory findings are resolved.

6.3 Normalization constraints

- Do not destructively remove punctuation or whitespace that can change code or secret structure.
- Maintain offset mappings between normalized text and original text.
- Use one documented indexing convention at internal interfaces; convert safely when external detectors differ.
- Treat zero-width characters and unusual Unicode as potential obfuscation signals, but avoid silently rewriting user text.
- Apply execution and input limits to avoid pathological regex behavior and denial-of-service conditions.

6.4 Deterministic detection rules

Rules should be independently testable and have stable identifiers, versions, severity defaults, confidence behavior, positive examples, negative examples, and documentation. A rule update must not silently change policy behavior without a version change.

- Email and supported phone formats.
- Supported government and financial identifier patterns.
- Recognizable provider-specific API keys and tokens.
- Authorization headers and bearer-token patterns.
- Password assignments and secrets in environment/configuration syntax.
- Private-key headers and structured cryptographic material.
- Database URLs and connection strings.
- High-entropy string heuristics used only as supporting evidence.
- Organization-defined keywords, identifiers, and patterns.

6.5 Optional NVIDIA/open-model support

The detector must expose an optional model-provider interface rather than depend directly on one model family. Candidate NVIDIA open models, including suitable Nemotron-family releases, may be evaluated for contextual classification. NeMo components may be considered where their actual license, runtime, and capabilities match the requirement. Model availability, open-weight status, license conditions, and hardware requirements vary by release and must be verified at selection time.

- Local inference is the default for any model-assisted detection.
- The model is advisory; deterministic detection and policy enforcement remain authoritative.
- Model outputs must pass strict schema validation and range validation.
- Scanned text is untrusted data, never trusted system policy or executable instructions.
- Benchmark model-assisted detection against the deterministic baseline on identical held-out data.
- Measure precision, recall, latency, memory/VRAM, and failure rate.
- Provide a configuration that disables models without disabling core protection.
- Do not transmit prompts to a hosted inference endpoint unless the user explicitly enables it after clear disclosure.

6.6 Confidence and severity

Confidence estimates the likelihood that a finding belongs to its category. Severity estimates potential harm if disclosed. A recognizable private-key block may have high confidence and critical severity; an ambiguous name may have low confidence and context-dependent severity. A token-like value must not be described as a confirmed live credential unless its status has been verified through an authorized, safe process. PromptVault must not validate secrets by attempting logins or contacting external services.

7. Risk scoring, policy decisions, and redaction

7.1 Initial scoring proposal

Use a deterministic, versioned scoring algorithm. A starting formula is:

```
R = min(100, max■(S■ × C■) + B)
```

Here, S is normalized severity (0–100), C is confidence (0–1), and B is a bounded bonus for combinations such as multiple credentials or credentials embedded in a connection string. The formula and coefficients are a proposal to be calibrated against evaluation data; they are not a validated security standard.

Score	Illustrative band	Default handling
0	No detected risk	Allow if scan completed and policy permits.
1–29	Low	Allow or review according to policy.
30–59	Medium	Recommend review/redaction; enforce configured rules.
60–84	High	Require remediation under strict policy.
85–100	Critical	Block under strict policy.

The score must not override mandatory category rules. A policy may block a private key regardless of aggregate score. Low-confidence findings should not be represented as confirmed disclosures.

7.2 Policy decisions

Decision	Meaning	Required behavior
ALLOW	No policy violation requires intervention.	Return decision and scan metadata.
WARN	Risk exists but policy permits continuation.	Return reasons and allow only where policy says so.
REDACT	Sensitive content must be sanitized.	Return safe candidate; validate before approval.
BLOCK	Disclosure is prohibited.	Prevent continuation in an enforced path.
REVIEW	A decision by an authorized actor is required.	Do not auto-allow while required review is unresolved.

7.3 Redaction modes

- **Masking:** replace a value with a generic typed marker, e.g. [REDACTED_EMAIL].
- **Typed placeholders:** use stable role-specific markers such as [API_KEY_1] or [EMAIL_1].
- **Consistent tokenization:** replace repeated instances of the same value with the same placeholder within a defined session scope.
- **Structured redaction:** where safe, preserve non-sensitive components of a connection string while removing credential fields.

7.4 Redaction invariants

- The original sensitive value must not appear in sanitized output unless explicitly retained by policy.

- Repeated values should receive consistent placeholders within the same scope.
- Distinct values must not be merged into one placeholder accidentally.
- Apply replacements using descending original offsets or a verified span-replacement method.
- Reject stale or overlapping spans unless a deterministic reconciliation policy resolves them.
- Re-scan sanitized output and report residual findings.
- Never automatically restore originals before external submission.
- A successful redaction operation does not guarantee the prompt is entirely free of sensitive information.

7.5 Optional restoration

Restoration is secondary and should remain out of the critical path for the MVP. If implemented, mappings must remain in transient local memory by default, never be sent to the model provider, and not persist across restarts unless the user explicitly enables a suitably encrypted mechanism. Restoration must be opt-in, must not automatically place secrets into AI chat inputs, and must validate placeholder provenance to avoid replacing attacker-controlled text.

8. Privacy, data lifecycle, and retention

8.1 Default data flow

User input → local validation → local detection → local policy evaluation → local sanitization → local post-redaction validation → result returned to the user.

The default standalone application should not transmit original prompt text, raw detected values, or reversible redaction mappings to a server. Hosting the web application's static assets does not require sending prompt content to the hosting provider. This property must be verified in network tests rather than inferred from architecture diagrams.

8.2 Data classification and handling

Data	Default handling	Persistence
Original prompt	Transient local processing only.	Not stored by default.
Raw detected value	Transiently referenced for span replacement.	Not stored by default.
Finding metadata	Category, offsets, severity, confidence, rule ID.	Transient; aggregate metadata may be stored only if explicitly enabled.
Sanitized prompt	Returned locally to user.	Not stored by default.
Placeholder mapping	Transient local session memory.	No persistence by default.
Security event	Outcome, category counts, policy ID/version, timestamp where enabled.	Optional, privacy-safe retention with documented expiry.
Diagnostic telemetry	Opt-in, minimized, no prompt contents.	Only under explicit policy and retention controls.
Model artifacts	Local runtime files selected by user/deployment.	Managed under software supply-chain controls.

8.3 Network egress requirements

- Core scanning must work when the application is offline after required assets are available.
- No prompt content may be sent to analytics, crash reporting, logging, or model endpoints by default.
- External model inference must be disabled by default and separately disclosed if ever offered.
- Any future cloud synchronization must be opt-in, documented, authenticated, and subject to a new threat review.

- Automated tests must verify the absence of prompt-bearing network requests during default operation.

8.4 Retention and deletion

Default retention for raw prompts, raw findings, and reversible mappings is zero beyond the active processing session. If optional event records are introduced, define their exact fields, purpose, access permissions, expiry period, deletion behavior, and export behavior before release. Logs must be reviewed for accidental inclusion of prompts, secrets, authorization headers, and stack traces containing input values.

8.5 Privacy limitations

Local processing reduces one class of exposure but does not protect against malware, compromised browsers, malicious extensions, screenshots, clipboard leakage, operating-system compromise, or disclosure to the destination AI service after the user submits a sanitized prompt. The product must not claim end-to-end confidentiality or guarantee the behavior of third-party AI providers.

9. System architecture and technology strategy

9.1 Logical components

Component	Responsibility	Constraint
Domain types	Shared finding, policy, decision, and scan-result contracts.	No browser or web-framework dependency.
Normalization engine	Input preparation and offset mapping.	Must preserve recoverable source spans.
Deterministic detector	Pattern rules, parsers, validators, custom rules.	Works without models or network.
Context provider	Optional local NER/model inference.	Pluggable, schema-validated, disabled independently.
Finding reconciler	Deduplication and overlap resolution.	Deterministic and tested.
Policy engine	Decision rules, mandatory blocks, severity/confidence thresholds.	No model-generated override of policy.
Redaction engine	Span replacement, placeholders, optional mapping.	No persistent original values by default.
Post-scan validator	Re-scan and verify policy resolution.	Must report residual risk.
Web application adapter	Connects the core engine to the standalone web product.	No direct server transmission of prompts by default.
Future browser adapter	Reads supported prompt inputs and enforces submission policy.	Minimum permissions; explicit supported-site scope.
Optional event recorder	Aggregate outcomes and privacy-safe events.	Disabled or minimized by default.

9.2 Recommended implementation direction

Area	Recommended option	Rationale / caveat
Web foundation	Next.js + TypeScript	Suitable for a web-first product; keep sensitive processing in client-side modules.
Code organization	Shared TypeScript security-core package	Enables reuse in web and future extension.
Validation	A schema validator such as Zod or an equivalent	Validate external/configuration/model outputs at boundaries.
Styling and UI	Implementation choice outside this PRD	No UI/UX specification is included in this document.
Pattern scanning	Custom rules plus vetted open-source detectors	Review maintenance, license, bundle size, and browser compatibility.
Presidio	Optional self-hosted or suitable local deployment	A remote Presidio service would violate local-only default unless explicitly disclosed.
NVIDIA models	Evaluate eligible open models locally	Confirm exact model license, weight availability, hardware, and benchmarks.
Database	None required for local MVP; optional store later	Avoid storing prompt content. Supabase or another service is not required for core protection.
Deployment	Static/client-capable deployment where practical	Verify that prompts never traverse application servers.
Testing	Unit, integration, property-based, adversarial, and browser tests	Security behavior requires more than happy-path testing.

9.3 Package boundaries

The security core should be a framework-independent package with pure or controlled functions for scanning, finding reconciliation, policy evaluation, and redaction. Model inference must be exposed through an interface such as ContextAnalyzer rather than imported directly throughout the codebase. Browser-specific code must be kept separate from the web adapter.

```
Suggested module boundaries: domain-types; normalization; detectors; context-analyzers;
findings; policy; redaction; validation; web-adapter; extension-adapter (future);
tests/fixtures.
```

9.4 Deployment modes

- **Local web mode:** client-side processing with no backend dependency for the core scan.
- **Optional self-hosted model mode:** model inference runs in a trusted local or organization-controlled environment; prompts are not sent to a third-party endpoint.
- **Future managed mode:** remote policy administration and aggregate reporting only after authentication, access control, privacy, and threat review.
- **Future extension mode:** local detection and policy enforcement within supported browser contexts, with explicit adapter health checks.

9.5 Technology selection gates

Before committing to a detector or model, verify license terms, maintenance status, package size, supported runtimes, browser compatibility, security history, hardware requirements, and benchmark results. Do not describe a model as open-source solely because weights are downloadable; inspect the exact release license and redistribution conditions.

10. Security threat model and abuse cases

10.1 Assets

- Original prompt text and embedded personal information.
- API keys, passwords, tokens, private keys, and database credentials.
- Placeholder-to-original mappings.
- Detection policies and organization-specific rules.
- Security event metadata and optional administrative configuration.
- Model artifacts, detector rules, build pipeline, and release signing material.

10.2 Trust boundaries

- User-provided prompt text is untrusted input.
- AI-generated output is untrusted and may contain misleading instructions or attacker-controlled text.
- Third-party AI services are outside PromptVault's direct trust boundary.
- Optional model inference is a separate component that may fail or produce invalid output.
- Browser content scripts operate in a hostile and changing webpage environment.
- Remote policy/configuration is trusted only after authentication, integrity validation, and schema validation.

10.3 Threats and mitigations

Threat	Impact	Mitigation / verification
Sensitive prompt leaks through telemetry	Privacy breach	No prompt telemetry by default; egress tests; log scrubbing.
False negative for a secret	Credential disclosure	Layered detectors, benchmark suite, strict policies, documented limits.
False positive blocks harmless text	Workflow disruption	Confidence-aware rules, negative tests, policy tuning, explainable findings.
Redaction range error	Secret remains or unrelated text is altered	Offset mapping, descending replacements, post-scan, property tests.
Prompt injection against local model	Misclassification or policy manipulation	Treat prompt as data; fixed schema; deterministic policy authority.
Malicious model output	Unsafe decision or crash	Schema validation, bounded output, no execution, fail-safe handling.
Malicious rule/config update	Policy bypass or data exposure	Authenticated updates, integrity checks, versioning, rollback.
Regex denial of service	High CPU or frozen scan	Safe patterns, input limits, execution budgets, fuzzing.
Browser adapter drift	Submission bypass	Health detection, adapter tests, strict-mode block when unavailable.
Extension over-permission	Unnecessary access to user data	Least-privilege manifest, permission audit, domain restrictions.
Placeholder collision/spoofing	Incorrect restoration	Unpredictable session-scoped IDs, provenance validation, opt-in restoration.
Credential validation side effect	External contact or credential misuse	Never test credentials against external services.
Supply-chain compromise	Malicious code or model artifact	Dependency pinning, scanning, provenance, reviewed updates.

10.4 Security modes

Permissive mode: policy may allow continuation after a warning or explicit decision. It is appropriate only where the user or organization accepts the residual risk.

Strict mode: mandatory findings, unresolved review requirements, and scanner failures block the enforced submission path. The system must not report successful protection if it cannot inspect the actual content that will be sent.

10.5 Security limitations

- No detector can guarantee zero false negatives.
- Semantic confidentiality is difficult to infer without context or organization-specific policy.
- Local execution does not protect against a compromised endpoint.
- A browser extension cannot guarantee coverage on unsupported websites or submission mechanisms.
- Third-party AI services may retain or process even sanitized prompts according to their own terms.
- A redacted prompt may still reveal sensitive information through context or inference.

11. API and internal contracts

The initial web application can call the security core directly without a network API. The contracts below define logical boundaries and can be implemented as TypeScript types or equivalent language-specific structures.

11.1 Scan request

```
ScanRequest { requestId: string text: string policyId?: string policyVersion?: string options?:
{ contextualAnalysis: boolean maxFindings?: number timeoutMs?: number } }
```

11.2 Finding

```
Finding { findingId: string category: string startOffset: number endOffset: number severity:
"low" | "medium" | "high" | "critical" confidence: number detectorId: string detectorVersion:
string reasonCode: string recommendedAction: "redact" | "review" | "retain" | "block" }
```

Confidence must be within [0, 1]. Offsets must be valid for the source string and use a documented indexing convention. Raw matched text should not be included in persisted findings by default.

11.3 Scan result

```
ScanResult { requestId: string status: "completed" | "partial" | "failed" findings: Finding[]
riskScore: number | null policyDecision: "allow" | "warn" | "redact" | "block" | "review"
reasonCodes: string[] detectorVersions: Record durationMs: number }
```

A failed or partial scan must not be represented as a clean scan. `riskScore` may be null if the scan cannot produce a trustworthy assessment.

11.4 Sanitization request and result

```
SanitizeRequest { requestId: string sourceText: string findingIds: string[] placeholderMode:
"typed" | "mask" } SanitizeResult { status: "completed" | "rejected" | "failed" sanitizedText:
string | null replacements: { findingId: string placeholder: string }[] residualFindingIds:
string[] validationPassed: boolean }
```

11.5 Error contract

Error code	Meaning	Required behavior
INPUT_TOO_LARGE	Input exceeds configured limit.	Reject without partial silent processing.
UNSUPPORTED_INPUT	Input type is not supported.	Report unsupported state.
SCAN_TIMEOUT	Detection exceeded its time budget.	Return failed/partial status; apply policy.
DETECTOR_FAILURE	A detector failed.	Report failure and apply configured fallback.
INVALID_FINDING_RANGE	A finding range is invalid.	Reject unsafe sanitization.
POLICY_INVALID	Policy failed validation.	Do not silently use a more permissive policy.
POST_SCAN_FAILED	Sanitized output could not be validated.	Do not mark output safe.

12. Non-functional requirements

12.1 Performance

Metric	Initial target	Measurement conditions
Deterministic scan latency	P95 ≤ 300 ms for a 10 KB prompt on a documented reference device.	Warm runtime; deterministic detectors only; record device/browser.
Sanitization latency	P95 ≤ 100 ms for 10 KB text after findings are available.	Includes replacement and validation overhead excluding full model inference.
Model-assisted latency	Set after model/hardware benchmark; must have a timeout and fallback.	Report CPU/GPU, model quantization, memory, and prompt length.
Input limit	Configurable; initial default 100 KB of text.	Reject or clearly report oversized inputs.
Memory usage	Measure and document for web runtime and any local model.	Test on reference hardware; avoid unbounded allocations.
Extension interception	No unbounded waiting; strict timeout policy.	Per-site end-to-end tests before release.

These are proposed engineering targets, not measured claims. Recalibrate them using the chosen detector set, browser, hardware, and test corpus.

12.2 Reliability

- Core detection must not depend on network availability.
- Every operation must return a defined success, partial, or failure status.
- Errors must not silently convert to ALLOW in strict mode.
- Optional model failure must leave deterministic detection operational.
- Policy parsing and redaction must be deterministic for identical input, rule versions, and configuration.
- Release builds must be reproducible to the extent supported by the toolchain.

12.3 Security

- Validate all external data, configuration, model output, and cross-context messages.
- Use least privilege and no unnecessary permissions.
- Do not execute prompt content, model output, or remotely supplied rules as code.
- Pin and audit dependencies; scan for known vulnerabilities.

- Protect any future administrative endpoints with authentication, authorization, rate limiting, and audit controls.
- Do not include secrets in test fixtures, logs, crash reports, source maps, or build artifacts.

12.4 Maintainability and portability

- Keep the security core independent of Next.js and browser-specific APIs.
- Version rules, policies, model adapters, and benchmark datasets.
- Document rule ownership and update procedures.
- Support model-disabled operation as a first-class configuration.
- Keep the extension adapter isolated from the core engine.
- Provide deterministic fixtures so results can be reproduced across environments.

12.5 Accessibility and internationalization

The product should support accessible operation and an extensible classification system. The initial detector corpus may focus on English and selected identifier formats, but supported languages and jurisdictions must be documented. Internationalization is a detection-quality problem as well as a translation problem: a translated label does not imply that contextual detection works in that language.

13. Testing, evaluation, and acceptance criteria

13.1 Test layers

Test layer	Coverage
Unit tests	Individual patterns, validators, span mapping, scoring, policy rules, placeholder generation.
Integration tests	Full scan → policy → redaction → post-scan flows.
Property-based tests	Range validity, deterministic replacement, unchanged unrelated spans, no accidental corruption.
Adversarial tests	Obfuscated tokens, unusual Unicode, misleading context, repeated secrets, prompt injection against model.
Privacy tests	Network egress, storage inspection, log scrubbing, no prompt content in telemetry.
Performance tests	Latency, memory, large inputs, pathological strings, model resource usage.
Regression tests	Known positives and negatives for every rule and model change.
Browser tests (later)	Actual outgoing content, keyboard/button submit, dynamic editors, timeout, unsupported adapter behavior.
Supply-chain tests	Dependency vulnerability scans, license checks, artifact integrity, build verification.

13.2 Evaluation dataset

Build a versioned, synthetic dataset with no real secrets or real personal records. Each case should include input text, expected category, expected character span, severity, expected policy outcome, and rationale. Include realistic positive and negative cases, repeated values, mixed categories, malformed credentials, code snippets, connection strings, and obfuscated variants.

- Generate fake email addresses and identifiers reserved for testing.
- Use synthetic credentials that cannot authenticate to any real service.
- Include harmless strings that resemble tokens to measure false positives.
- Include negative examples where sensitive words appear without sensitive values.

- Keep train/development cases separate from held-out evaluation cases.
- Version every dataset release and record detector/model versions with results.

13.3 Metrics

Metric	Definition	Why it matters
Precision	TP / (TP + FP)	Measures how often reported findings are correct.
Recall	TP / (TP + FN)	Measures how many expected findings are detected.
F1	Harmonic mean of precision and recall	Summarizes trade-off but must not replace separate reporting.
Span accuracy	Correctly localized finding spans / expected spans	Redaction must target the correct characters.
Policy decision accuracy	Correct policy outcomes / evaluated cases	Measures enforcement behavior.
Redaction success	Cases where required values are removed and output validates / required redactions	Measures end-to-end sanitization.
False block rate	Harmless cases blocked / harmless cases tested	Measures avoidable workflow disruption.
Latency	P50/P95 scan and sanitization duration	Measures operational usability.
Egress violations	Prompt-bearing network requests in default mode	Must be zero in tested default flows.

13.4 Proposed release acceptance criteria

- All P0 requirements have corresponding passing tests or documented, approved exceptions.
- No original prompt or raw detected value is transmitted during tested default web-app flows.
- The core scan operates without a paid AI API and with model inference disabled.
- All high-confidence synthetic credential cases in the release acceptance corpus are detected; any misses are investigated and documented.
- Precision and recall are reported separately by category; no single aggregate score hides weak categories.
- Every redaction test verifies both that the target value is removed and that unrelated text is preserved.
- Sanitized output is never marked validated if post-scan fails.
- Strict policy blocks on scanner failure for enforced operations.
- The model-assisted path, if included, demonstrates measurable benefit over baseline and passes schema/adversarial tests.
- Known limitations, supported categories, model license, and deployment assumptions are documented.

The dataset and exact numeric detection thresholds should be frozen before the release evaluation. A claim such as '100% detection' must be limited to the exact named test corpus and must not be presented as universal real-world coverage.

14. Metrics and success criteria

14.1 Product metrics

Metric	Initial target / approach
Core flow completion	At least 95% of valid test inputs complete without an unhandled exception.
Synthetic high-confidence credential recall	Target ≥ 95% on a frozen, representative acceptance corpus; report per credential family.
PII precision	Set category-specific target after baseline measurement; publish precision and recall separately.
Sanitization correctness	100% of acceptance cases must remove selected spans and preserve designated unaffected spans.
Default-mode prompt egress	Zero prompt-bearing requests in tested workflows.
Model value	Adopt only if it improves an agreed metric without unacceptable latency or privacy cost.
Extension enforcement (later)	Every supported adapter passes defined positive, negative, failure, and bypass tests.

These targets are proposed release gates and should be revised transparently if initial measurements show that they are not achievable without compromising reliability.

14.2 Hackathon evidence

- A live scan of a synthetic prompt containing multiple sensitive categories.
- Explainable findings with category, confidence, severity, and reason codes.
- Redaction that preserves surrounding prompt meaning and structure where possible.
- Post-redaction rescan showing which findings were resolved and any residual risk.
- An attack simulation suite with measurable true/false positive and negative counts.
- Evidence that default processing does not send original prompt content to an external service.
- A concise limitations statement that distinguishes a working prototype from a production security certification.

15. Deployment, operations, and governance

15.1 Build and release

- Use a protected main branch and require automated tests for security-critical changes.
- Pin dependencies and review changes to detection rules, policy defaults, and model artifacts.
- Run dependency and secret scans in continuous integration.
- Generate a release artifact with a documented version and changelog.
- Keep development, test, and production configuration separate.
- Do not embed secrets in client bundles; client-side environment variables are not secret storage.
- Provide a rollback path for faulty rule or model releases.

15.2 Rule governance

Each rule must have an owner, identifier, version, rationale, positive tests, negative tests, severity default, confidence behavior, and change history. Changes that increase or reduce blocking behavior require explicit review. Rule updates must not silently relax mandatory policies.

15.3 Model governance

- Record exact model name, revision, license, source, checksum/provenance where available, runtime, and quantization.
- Review permitted use, redistribution, and commercial/hackathon use terms for the exact release.
- Benchmark before upgrading model weights or inference libraries.
- Keep deterministic fallback and a kill switch for model inference.
- Never allow model output to alter trusted policy or execute code.
- Document resource requirements and whether model files are downloaded separately.

15.4 Incident handling

If a release is found to leak prompts, misapply redaction, or bypass strict policy, disable the affected functionality, preserve privacy-safe diagnostic information, reproduce the issue with synthetic data, ship a reviewed fix, and publish a clear incident note. Do not collect real sensitive prompts merely to debug a detection failure.

15.5 Future organizational deployment

Team policies, managed configuration, and aggregate reporting are later-stage capabilities. Before adding them, define tenant isolation, authentication, authorization, policy integrity, administrative audit records, retention, deletion, and data-residency expectations. Do not add a central prompt archive as a default enterprise feature.

16. Roadmap and workstreams

Phase	Workstream	Deliverables	Exit gate
1	Domain and contracts	Finding/policy/result types, error contract, versioning.	Contracts reviewed and unit-tested.
1	Detection core	Normalization, rules, validators, overlap resolution.	Synthetic positives and negatives pass.
1	Policy/redaction	Risk score, decisions, placeholders, post-scan.	End-to-end redaction tests pass.
1	Web application	Standalone client-side integration and local processing.	Default egress verified; core works offline after load.
1	Evaluation/demo	Synthetic corpus, attack scenarios, metrics report.	Repeatable demo with recorded results.
2	Hardening	Fuzzing, performance, dependency and privacy checks.	Release gate passes or exceptions documented.
3	Extension core	Manifest V3 scaffold, adapter contract, local engine reuse.	Minimum permissions reviewed.
3	Site adapters	Supported website interception and failure handling.	Per-site bypass and submission tests pass.
4	Team deployment	Authenticated policy distribution and aggregate reporting.	Threat model and access control reviewed.

16.1 Suggested team ownership

Workstream	Responsibilities
Security core	Detection rules, finding schema, scoring, policy engine, redaction, tests.
Web product	Standalone application integration, offline behavior, error handling, deployment.
Model evaluation	Candidate NVIDIA/open-model review, local inference adapter, benchmark comparison.
Security QA	Threat model, synthetic corpus, adversarial tests, privacy egress tests, release checklist.
Extension (later)	Manifest, permissions, site adapters, submission interception, compatibility testing.

16.2 Dependency order

The detector and policy contracts should be agreed before parallel work expands. Redaction depends on stable finding offsets; the web integration depends on stable scan/result contracts; model integration depends on the contextual analyzer interface; the extension depends on a framework-independent core and a validated enforcement policy.

17. Hackathon demonstration plan

The demo should prove a complete defensive workflow rather than rely on slides or an unverified claim of AI security.

1. **Baseline:** scan harmless text and show that the system completes a scan without inventing a high-confidence secret.
2. **Credential exposure:** scan synthetic source code containing a fake API key and a database password.
3. **PII exposure:** scan synthetic text containing fake contact information and a configured internal identifier.
4. **Redaction:** apply typed placeholders, preserving unrelated code and prose.
5. **Verification:** rescan sanitized output and show resolved findings plus any residual risk.
6. **Adversarial case:** test a token-like false positive, unusual separators, or Unicode variation and explain the detector's outcome.
7. **Model comparison:** if a local model is integrated, compare baseline versus model-assisted results on the same held-out cases, including latency and errors.
8. **Privacy evidence:** demonstrate the absence of prompt-bearing network requests in default mode using browser/network instrumentation.
9. **Failure mode:** simulate detector failure and show that strict policy does not silently mark the prompt safe.

17.1 Demo rules

- Use only synthetic, nonfunctional credentials and fake personal data.
- Do not paste real API keys, customer records, or private documents into the demonstration.
- Do not claim to have verified a credential's live status.
- Show the actual tested behavior and limitations; do not simulate successful interception without a working implementation.
- If the model is too slow or unreliable for the demo environment, demonstrate the deterministic baseline and describe the model as experimental.

18. Risks, assumptions, dependencies, and open decisions

18.1 Risks and mitigations

Risk	Likelihood / impact	Mitigation
Detection misses unfamiliar secrets	Medium / high	Layered rules, contextual detection, synthetic benchmark, clear limitations.
False positives reduce trust	Medium / medium	Negative cases, confidence thresholds, configurable policies.
Local model requires excessive hardware	Medium / medium	Keep model optional; benchmark smaller/quantized candidates; deterministic fallback.
Model license is unsuitable	Low–medium / high	Review exact release terms before adoption; retain provider abstraction.
Browser sites change	High / high for extension	Explicit adapter health, per-site tests, strict-mode failure behavior.
Prompt content leaks through logs	Low / critical	Data-minimizing architecture, egress/logging tests, no raw content by default.
Scope exceeds hackathon time	High / medium	Web-first delivery; extension deferred; model integration only after baseline.
Redaction damages prompt meaning	Medium / medium	Typed placeholders, output validation, preserve unaffected spans, disclose limits.
Performance degrades on long inputs	Medium / medium	Input caps, timeouts, regex safety, performance benchmarks.

18.2 Assumptions

- The first release is a standalone web application with client-side security processing.
- The browser extension is a later release, not a requirement for the first complete MVP.
- No paid AI API is required for core protection.
- Optional NVIDIA/open models may be used after license and technical evaluation.
- The initial corpus is synthetic and avoids real personal data or operational secrets.
- A database is unnecessary for core local scanning; future team features may introduce one.

18.3 Open decisions to resolve during implementation

1. Which exact sensitive-data categories and jurisdictions are in the first benchmark corpus?
2. Which secret-scanning library, if any, offers the best balance of coverage, license, maintenance, and browser compatibility?
3. Will contextual NER be included in the first web MVP, or deferred until deterministic detection is benchmarked?
4. Which exact NVIDIA/open model, if any, meets license, latency, memory, and quality requirements on the intended hardware?
5. What default policy should apply to ambiguous PII versus high-confidence credentials?
6. What input-size and latency limits are acceptable for the first release?
7. Which AI websites will be explicitly supported by the first extension release?
8. Will optional event history be included, and if so, what exact retention and deletion rules apply?

Appendix A. Sample policy and sample findings

A.1 Illustrative policy

```
policyVersion: "1" mode: "strict" rules: - category: "private_key" action: "block"
minConfidence: 0.80 - category: "credential" action: "redact" minConfidence: 0.75 - category:
"email" action: "warn" minConfidence: 0.80 - category: "custom_confidential" action: "review"
minConfidence: 0.70 onScanFailure: "block" postRedactionValidationRequired: true
```

This is illustrative, not a universal recommended policy. Exact thresholds must be calibrated. A mandatory block must remain authoritative regardless of the aggregate risk score.

A.2 Illustrative finding

```
{ "findingId": "f-001", "category": "credential", "startOffset": 42, "endOffset": 67,
"severity": "critical", "confidence": 0.96, "detectorId": "secret.generic-token",
"detectorVersion": "1.0.0", "reasonCode": "TOKEN_PATTERN_AND_CONTEXT", "recommendedAction":
"redact" }
```

The example contains no actual secret. In persistent records, the raw matched value should be omitted by default.

A.3 Example sanitized prompt

```
Original: Please debug this service.
DATABASE_URL=postgres://admin:fake-password@db.example.test/app Sanitized: Please debug this
service. DATABASE_URL=[DATABASE_CONNECTION_STRING] Expected result: - The credential-bearing
connection string is not present in sanitized output. - The finding is recorded as resolved only
after post-scan validation. - The sanitized prompt may still contain other sensitive information
not detected by current rules.
```

Appendix B. Glossary

Term	Definition
DLP	Data loss prevention: controls intended to identify and reduce unauthorized disclosure of sensitive data.
PII	Personally identifiable information, interpreted according to the relevant context and policy.
Secret	A credential or sensitive value such as a token, password, private key, or access key.
Finding	A detector's structured claim that a text span may represent sensitive information.
Confidence	Estimated likelihood that a finding is correctly classified.
Severity	Potential impact if the finding is disclosed.
Policy decision	The action returned after applying configured rules to findings and context.
Redaction	Replacing sensitive text with a marker or placeholder.
Post-scan	A second scan of sanitized output to check whether required findings remain.
False positive	A reported finding that does not match the intended sensitive category.
False negative	A sensitive item in the evaluation ground truth that the detector misses.
Local inference	Model inference performed on the user's device or a trusted local/organization-controlled environment.
Fail closed	Deny the protected operation when a required security check cannot be completed.
Adapter	A platform-specific integration that connects the shared security core to a host environment.

Final product requirement

PromptVault must be built as a web-first, privacy-first security product with a reusable detection and policy core. The initial release must demonstrate measurable, reproducible protection without paid AI APIs. Optional open models may improve contextual detection only when they meet explicit privacy, licensing, performance, and evaluation gates. Browser extension support follows after the standalone application and its security guarantees have been validated.

End of document.